

Project Discovery and Hierarchical Data

Kirill Nevzorov under supervision
of Prof. Christopher Buckley
University of Sussex
School of Engineering and Informatics

April 23, 2026

Abstract

This report presents the design and implementation of an LLM-powered developer assistant built with a Quarkus backend and a web frontend, and evaluates it through an agentic model analysis lens. The central value of the system lies in its workflow: enabling a developer to query project history, explore repository state, and interrogate git artefacts through a natural-language interface rather than through manual inspection of command output. Beyond workflow convenience, the project demonstrates that structuring project knowledge as hierarchically organised package-level data improves modularity and maintainability, making it easier to isolate responsibilities, evolve components incrementally, and reason about the implementation impact of new features across the codebase.

The implementation applies conventional software engineering practices, including CDI-managed dispatch, transaction-scoped persistence, and iterative refinement through staged development and testing. Findings are interpreted as constrained engineering evidence, with emphasis on reliability, orchestration behavior, and failure patterns rather than universal benchmark claims. The final build represents a partial realisation of the intended system: Quarkus build-time CDI compilation introduced packaging-stage regressions, and the toolset remained constrained to a minimal working subset. The report therefore presents both the practical strength of bounded agentic workflows and an honest account of the remaining gap between intended design and delivered artefact.

Contents

1	Introduction	4
1.1	Research Questions	4
1.2	Objectives	4
1.2.1	Strand A: Technical Delivery	4
1.2.2	Strand B: Workflow Evaluation	4
1.2.3	Objective-to-Evidence Mapping	4
1.3	Report Structure	5
2	Overview	5
2.1	Coding Agents	5
2.1.1	VS Code Copilot	5
2.1.2	VS Code Copilot with Local Ollama Integration	5
2.1.3	Claude Code	6
2.1.4	Ollama (Model Runtime)	6
2.1.5	LM Studio	6
2.1.6	msty.ai	6
2.2	Vim	6
2.3	Java with Quarkus	6
2.3.1	Features Considered for Implementation	6
2.3.2	Key Libraries	7
2.4	Professional Considerations	7
2.5	Findings and Achievements	7
3	Background	7
3.1	The Role of LLMs in Software Development	8
3.1.1	Key Capabilities of LLMs in Coding	8
3.1.2	Integration with Engineering Workflows	8

3.1.3	Current Research on LLM-Assisted Software Engineering	8
3.2	Motivation	8
4	Project Description	9
4.1	Scope	9
4.1.1	In Scope	9
4.1.2	Out of Scope	9
4.1.3	Practical Constraints	9
4.2	Methodology	9
4.2.1	Phase 1: Problem Framing and Requirements	9
4.2.2	Phase 2: Architecture and Implementation	10
4.2.3	Phase 3: Integration and Verification	10
4.2.4	Phase 4: Evaluation and Reporting	10
4.2.5	Agentic Evaluation Protocol	10
4.2.6	Evaluation Perspective	11
5	Project Requirements	11
5.1	Capability Contract	11
5.1.1	Capability Matrix	11
5.1.2	Design Principles	12
5.2	Functional Requirements	12
5.2.1	Model Selection and Tool-Calling Support	12
5.2.2	Hardware Constraints and Multi-Host Benchmarking	12
5.2.3	Model Runtime Requirements	12
5.2.4	Development Environment RAM Requirements	13
5.2.5	Benchmarking and Verification Framework	13
5.2.6	Testing and Verification	13
5.2.7	Multi-Provider Support	14
5.3	Non-Functional Requirements	14
5.3.1	Maintainability	14
5.3.2	Spec Boundary Management	14
5.4	Deferred Requirements	14
6	Project Architecture	14
6.1	Application Structure	15
6.1.1	Understanding Large Language Models	15
6.2	End-to-End Runtime Flow	15
6.3	Package Map	15
6.3.1	Core Components	16
6.4	Services Layer	17
6.5	Architectural Choices	17
6.5.1	Error and Signal Model	17
6.5.2	Persistence and Context Caching	17
6.5.3	System Integration	17
7	Requirements Gathering: AI/LLM Integration (Draft)	18
7.1	Introduction	18
7.1.1	Mandatory Review	18
7.1.2	Foreshadowing Context	18
8	Implementation Timeline	18
8.1	Milestone Reference	18
9	Results and Discussion	19
9.1	Evaluation Lens	19
9.2	Key Findings	19
9.3	Evidence Matrix	19
9.4	Quantitative Evaluation Targets	20
9.5	Discussion	20
9.6	Limitations	20
9.7	Threats to Validity	21
9.8	Testing and Evaluation	21

9.9	Future Work	21
10	Conclusion	21
11	Appendices	23
11.1	[-] Prompt Format Note: Observed Critical Usage Pattern	23
11.2	Appendix: Supporting Scripts and Code Artifacts	23
11.3	A. System Diagrams and Architecture Charts	23
11.3.1	A.1 Tool Execution Sequence	23
11.4	C. Frontend Workspace Screenshots	24

1 Introduction

This report investigates how locally hosted Large Language Models (LLMs) can be integrated into a software system for project inspection and repository analysis under practical engineering constraints. The implemented artefact, PDHD (Project Discovery and Hierarchical Data), combines a Quarkus backend, a web frontend, and tool-mediated interaction with local models to support filesystem exploration, file analysis, and project-level summarisation.

The work addresses two linked concerns. First, can a locally deployed LLM-assisted architecture produce useful, evidence-grounded project summaries without relying on hosted APIs? Second, how effective is agentic coding as a development workflow when measured against conventional software engineering concerns such as maintainability, validation effort, and operational reliability?

To evaluate these questions in a realistic setting, the system was built around local inference. Ollama was used as the model runtime, with two NVIDIA RTX 3060 GPUs (24GB combined VRAM) defining the practical resource envelope for model selection and execution.

The report therefore treats LLMs in a dual role: as runtime components inside the application, and as development assistants during design and implementation. Rather than presenting only implementation progress, the report aims to evaluate the architecture and workflow against explicit objectives, constraints, and evidence.

1.1 Research Questions

1. To what extent can a local LLM-assisted system generate grounded and practically useful project-inspection outputs?
2. Which architectural decisions most strongly affect reliability and maintainability in tool-calling LLM workflows?
3. What benefits and limitations emerge when agentic coding is used as a primary development method?

1.2 Objectives

The project objectives are organised into two strands: technical delivery and workflow evaluation. The initial scope was revised from low-level resource governance to application-level project inspection so that the implementation effort remained aligned with the report’s central research questions.

1.2.1 Strand A: Technical Delivery

1. Deliver a deployable web-based system for repository inspection and summarisation using local LLM inference.
2. Implement tool-calling support for bounded project operations (for example file reading and directory inspection).
3. Provide persistent project knowledge so analysis can be revisited and updated.

1.2.2 Strand B: Workflow Evaluation

1. Characterise how agentic coding affected implementation speed, structure, and rework.
2. Evaluate whether generated outputs remained aligned with developer intent and engineering constraints.
3. Identify limitations and failure patterns that constrained reliability.

1.2.3 Objective-to-Evidence Mapping

Objective	Evidence expected in report	Primary indicators
Deployable local LLM-assisted system	Architecture description, integration notes, and runnable workflow	End-to-end scenario completion under local runtime constraints
Reliable bounded tool-calling	Tool execution examples, observed failures, and mitigation strategy	Tool-call validity, categorized failure frequency
Persistent project knowledge	Storage design, retrieval workflow, and recall examples	Successful retrieval and reuse across repeated analysis runs
Evaluate agentic coding process	Development reflections tied to concrete implementation episodes	Retry/correction burden, workflow friction patterns
Assess quality and maintainability	Discussion of rework, simplification, and boundary enforcement	Boundary clarity, regression profile, maintainability trade-offs

Objective	Evidence expected in report	Primary indicators
Identify constraints and limitations	Explicit limitations and threats-to-validity sections	Bounded-claim statements tied to environment and sampling limits

1.3 Report Structure

This report comprises three phases: foundational context, design planning, and outcomes. The Introduction frames the research questions and the value proposition of evidence-grounded project inspection through natural-language workflows. Background contextualises LLMs, agentic coding, and known reliability constraints from current literature. Project Description outlines the problem domain, functional requirements, and the evaluation protocol used to analyse agentic model behavior in this implementation context.

Implementation and outcome sections ground design in practice. Project Architecture describes the layered backend, frontend signal-based orchestration, persistence strategy, and tool-calling boundaries that shape agentic behavior. Implementation Timeline provides a Gantt overview of planned development phases and delivery shifts. Results and Discussion present observed outcomes, scenario-level analysis, validity threats, and gaps between intended design and delivered artefact, distinguishing measured findings from interpretation. The conclusion closes each objective strand against available evidence and identifies prioritized next steps.

2 Overview

This section introduces the core technologies used in the project and clarifies how they were combined into a practical development workflow. The setup is presented in two parts: the developer environment and the LLM environment.

The developer environment is centered on Visual Studio Code, with Debian running under WSL as the primary host context for day-to-day implementation work. Developer environment and tooling is managed through Docker with devcontainers to keep dependencies consistent and reproducible across sessions.

Ollama, LM Studio, and msty.ai model servers on the homelab workstation were evaluated for integration with editor-based assistants including VS Code Copilot and Claude Code. These tools proved essential for rapidly developing features and estimating feasibility within available project time.

Heavy use of terminal scripting combined with Vim-style keybindings and Makefile workflow served as an additional productivity factor in this setup within Visual Studio Code. These made the workflow strongly keyboard-driven: routine tasks such as running builds, restructuring files, searching project-wide text, and iterating on report output could be executed with minimal context switching. In practice, this interaction style reduced friction during frequent edit-build-review cycles and supported faster iteration when coordinating code changes, tooling commands, and documentation updates.

2.1 Coding Agents

This subsection compares the coding-agent and local model tooling used during implementation, with attention to practical developer ergonomics, task fit, and observed operational constraints.

2.1.1 VS Code Copilot

VS Code Copilot served as the primary development assistant in this project. It was used extensively to accelerate implementation tasks that would otherwise require repeated documentation cross-referencing, and it generally provided a comfortable and reasonably ergonomic workflow with usable default keybindings and acceptable agentic guardrails. Typical delegated work included rapid frontend React development, coordinated API updates across backend and frontend components, and ongoing documentation maintenance through markdown-oriented subagent workflows. In practice, the available token budget was a material constraint: the free tier was insufficient.

An explorative build-rewrite-analyse workflow implementation required the Copilot Pro Plus subscription.

2.1.2 VS Code Copilot with Local Ollama Integration

VS Code Copilot connected to a local Ollama backend was evaluated for the same classes of tasks as standard Copilot usage. Although this setup reduced direct subscription cost, it depended on powering an auxiliary workstation for each session and did not provide adequate responsiveness for routine development. Once the

higher Copilot subscription tier was adopted, the local integration path became functionally redundant for day-to-day work.

2.1.3 Claude Code

Claude Code was tested through a local configuration approach in which the base URL was manually redirected to an Ollama instance. This setup was explored primarily for personal productivity tasks, including hardware-inventory maintenance and draft eBay listing generation, rather than core project delivery. The main observed limitation was context-window pressure relative to available workstation resources, which restricted usefulness despite a polished terminal interface.

2.1.4 Ollama (Model Runtime)

Ollama functioned as a central model runtime for the application and, in some sessions, for agentic coding support. Model selection converged on Gemma4 because it offered a favorable speed profile while still supporting a large context window and CPU plus system-RAM offloading. Other attempted models included multiple Qwen3 sizes, glm-4.7-flash, gpt-oss, Qwen2.5-coder, Llama3.2, and Llama3.1. A consistent practical finding was that local inference quality and throughput depended strongly on careful model selection; a longer project timeline would likely have enabled a more rigorous benchmarking environment for direct model-to-model comparison.

2.1.5 LM Studio

LM Studio was most effective as a question-answering interface in this workflow. Its development-server port mode, intended to expose the bundled model runner over the network, was constrained by binding behavior to localhost:11434, which prevented connections from the devcontainer and remote machines. Relative to msty.ai, its folder and conversation organization workflow was easier to use, though less feature-rich.

2.1.6 msty.ai

msty.ai was also used primarily for question answering. It provided support for Ollama and llama.cpp, with explicit backend options across different GPU vendors and CPU execution. In this project context, these backend controls were not essential because the dedicated Ollama server already handled automatic GPU/CPU layer offloading across available processing units. Compared with other interfaces, msty.ai offered broad capability but a noisier, less developer-ergonomic user experience.

2.2 Vim

Vim-style editing was central to this workflow, supported by custom keybindings: **Ctrl-A** (rebind of **Shift-A**), **Ctrl-B** (terminal management), **Ctrl-G** (file management), and **Ctrl-I/Alt-I** (agent management). These mappings made routine typing and editing operations significantly more fluid and reduced interruption during implementation.

VS Code Copilot also integrated effectively as inline autocomplete. Useful suggestions and refactoring actions were commonly accepted directly with a **Tab** keypress, avoiding repeated navigation to the refactoring menu. However, suggestions occasionally lagged behind developer input, despite offering faster iteration than manual refactoring approaches.

2.3 Java with Quarkus

2.3.1 Features Considered for Implementation

At the start of this project, I expected to explore a wider implementation space than what would eventually fit into the final report. I planned for support of multiple LLM providers, including internal Testcontainers-based options and external Ollama base-url based options, so that the system could stay usable under different runtime conditions. I also wanted to keep a native deployment path available for lower-footprint execution and to include workstation-oriented integration testing for real model and tool interactions.

During development I explored different higher-level usability goals, including nested menu and assistant session flow through declarative coding, clean entry and exit behavior, and predictable returns to parent contexts. In parallel, I focused on seeding the frontend codebase with strong design decisions, including packaging the Web UI as a React application with modules and a signals library implemented as separate logical packages.

2.3.2 Key Libraries

The implementation stack is built on Quarkus extensions and complementary libraries. The following summarises the core runtime dependencies:

This produces the complete hierarchical view of transitive dependencies, pinned versions, and any convergence warnings. These libraries formed a practical foundation for implementing the planned capabilities within the Quarkus runtime model.

2.4 Professional Considerations

I have developed this project by applying the knowledge and skills acquired throughout the Computer Science modules (G400) taught at the University of Sussex. In considering my professional responsibilities, I have used the [BCS Code of Conduct](#) as a supporting framework. In particular, I have sought to ensure that any evaluative claims made about software systems are presented carefully and responsibly, consistent with section 2.f, which requires members to avoid injuring others, their property, reputation, or employment through false, malicious, or negligent action or inaction.

This research does not involve the collection of user data or other sensitive information, because it is based on the analysis of software and related materials that are already in the public domain. The discussion is intended to contribute constructively to professional practice, in line with section 4.e of the BCS Code of Conduct, which states that members should encourage and support fellow members in their professional development. This also aligns with the Code's wider expectation that continuing professional development should broaden knowledge of the IT profession and help maintain professional competence.

2.5 Findings and Achievements

This project pursued higher-order workflow capabilities through a specification-as-documentation approach rather than isolated endpoint implementations. The implementation applied functional programming principles, test-driven development practices, and Git-based Agile workflows to realise menu-driven interaction models, modular tool orchestration across exploration, reading, writing, introspection, and persistent project knowledge management, alongside Retrieval-Augmented Generation with reproducible results.

The application stack supports practical reliability and governance features, including transactional execution boundaries, explicit error pathways, telemetry-backed observability, and compatibility handling for evolving tool interfaces.

On the frontend, it supports progressive frontend integration for browsing, summarisation, and activity tracing, with discoverability and UX gaps easier to identify and address iteratively. Overall, a platform that could be refined continuously under real development constraints was produced.

Investigation of Quarkus capabilities showed promise for iterative design, though some features—automated model provisioning, fallback startup paths, startup-time REST client wiring, and strict config mapping—proved reliable only under specific conditions. This required a more cautious approach based on explicit validation, fail-fast behavior, and incremental compatibility adjustments.

3 Background

Recent industry developments indicate that LLM-enabled software engineering has moved from exploratory experimentation toward mainstream technical and commercial relevance. Public discussion around agentic coding systems, local model serving, and AI infrastructure has accelerated, with signals such as the moltbot/openclaw discourse, Anthropic's agentic software bounty framing, and NVIDIA's profitability trajectory often used to illustrate momentum. In this report, however, these social and news signals are treated as contextual indicators of pace and attention rather than as primary technical evidence.

Within that context, this project examines how LLM-based workflows help build and reshape engineering workflows, and why local practical experimentation is a meaningful object of study. The aim is not to reproduce hype-driven claims, but to evaluate implementation outcomes under real constraints, including tooling volatility and rapidly changing practices. Accordingly, the report grounds its evidence in build decisions, observed system behavior, and structured evaluation.

3.1 The Role of LLMs in Software Development

LLMs can support software development by translating intent into explanations, structured documentation, and code. Their value in modern engineering work can be seen in the shift from repetitive drafting toward evaluation, integration, and quality control activities.

Within that framing, two related modes are useful to distinguish. In LLM-assisted development, the developer remains the primary decision-maker and uses models to draft code, investigate errors, write documentation, and propose refactorings. Agentic development extends this approach by allowing the model to execute bounded multi-step tasks, such as planning actions, invoking tools, and iterating toward a defined objective under human supervision. Together, these modes can shorten iteration cycles and expand exploratory capacity, while increasing the need for validation, architectural discipline, and constraint-aware decision making.

3.1.1 Key Capabilities of LLMs in Coding

Code Generation: LLMs can draft snippets, components, and implementation skeletons from requirements.

Debugging Assistance: They can propose fault hypotheses, explain error traces, and suggest candidate fixes.

Documentation: They can accelerate production of comments, API notes, and technical summaries.

Refactoring Support: They can suggest structural changes that improve readability and maintainability.

When applied in bulk, these operations can often outperform a purely human approach, as the AI model can approach problems from multiple endpoints at the same time, or perform remote edits by transforming batches of files while the developer remains focused on the current problem scope.

Caution is needed when LLM tooling is less accurate than the specification the developer provides, as it happens with any software. A key symptom is having to rework large amounts of quickly generated code with loosely specified intent, remove unnecessary abstractions, and unpack overly verbose documentation.

3.1.2 Integration with Engineering Workflows

In practical workflows, these capabilities are often delivered through editor assistants, CLI agents, and service integrations that operate during active development. Tools such as GitHub Copilot can provide real-time suggestions, while more agentic systems can execute multi-step tasks across files and tools under developer control.

3.1.3 Current Research on LLM-Assisted Software Engineering

Recent software engineering literature reports that LLM-assisted development can improve throughput in selected tasks, but that reliability, evaluation quality, and reproducibility remain unresolved concerns in many practical settings (Fan et al. 2023; Hou et al. 2024). The current consensus is therefore cautious rather than absolute: these systems are most effective when used with explicit constraints, verification steps, and measurable quality criteria instead of anecdotal performance claims.

Recent work on agentic systems further suggests that multi-step tool use introduces additional coordination and control challenges, especially when systems move from single-turn drafting into workflow-level automation (Plaat et al. 2025). For code-focused pipelines, retrieval-augmented approaches are increasingly studied as a way to ground generation in repository context and reduce unsupported outputs (Kivroglou, Schlippe, and Martin 2025; Wang, Guo, and Tan 2025).

For this report, the focus is on how reliably the output can be integrated into disciplined software engineering processes without reducing verification, maintainability, or accountability.

3.2 Motivation

Current industry attention to LLM-enabled software development makes this project timely in both practical and academic terms. The rapid uptake of coding assistants, sustained discussion of agentic tooling, and frequent updates across vendor ecosystems indicate that workflows are changing quickly enough to warrant close, implementation-level examination. This report uses these developments as contextual justification for the studied technological gap and problem relevance; they are not treated as primary evidence for technical conclusions.

Software engineering remains the core discipline: architecture, implementation, testing, and integration are all treated as essential and non-negotiable. The challenge in an academic setting is timeline compression: scope and schedule are often fixed before implementation begins, and meaningful results must be produced within months rather than years.

Therefore, the LLM-assisted workflow was chosen as a rapid prototyping strategy, while retaining responsibility for technical decisions, quality control, and final evaluation. An additional pass was also performed over the project documentation in order to produce a timeline graphic using terminal and command-line tooling, as accessibility constraints made most modern design and graphing software impractical.

4 Project Description

Software repositories often contain enough implementation evidence to infer project purpose, maturity, and likely next steps, yet this evidence is distributed across files and difficult to inspect quickly. PDHD (Project Discovery and Hierarchical Data) was designed as a project-inspection system to address this problem by combining filesystem exploration, tool-mediated analysis, and LLM-assisted summarisation.

The chapter frames PDHD as both an implemented software artefact and a constrained engineering study. The implementation context is intentionally practical: local model execution, limited hardware, and incremental integration into a real development workflow. Within those boundaries, the project contributes a structured approach to repository analysis that emphasises explicit tool boundaries, persisted project knowledge, and evidence-grounded output generation.

The following sections define scope and methodology so that subsequent architecture and results chapters can be interpreted against clear boundaries rather than retrospective narrative.

4.1 Scope

The scope is intentionally bounded to keep implementation and evaluation aligned.

4.1.1 In Scope

1. Build a web interface for file system analysis
2. Implement LLM-driven file content summary service
3. Document the development workflow and challenges
4. Conduct substantial internal evaluation of agentic workflow behavior across representative scenario classes
5. Report reliability, latency, and failure patterns using traceable evidence from this implementation context

4.1.2 Out of Scope

1. Full autonomous software engineering without human validation
2. Production-grade completion scoring across arbitrary repositories
3. Large-scale benchmark evaluation across many independent projects
4. Fine-tuning or training bespoke language models
5. General claims of model superiority outside this project's environment and task profile

4.1.3 Practical Constraints

1. Local inference runtime and available hardware constrained model choice and throughput.
2. Tool coverage remained intentionally bounded to selected project-inspection operations.
3. Evaluation evidence is drawn from this implementation context and is not intended as a universal benchmark claim.

4.2 Methodology

The project follows a design-and-evaluation methodology tailored to a practical engineering artefact. The process combines implementation work with structured reflection on architectural outcomes and workflow behaviour, with specific emphasis on agentic model analysis under constrained local-runtime conditions.

4.2.1 Phase 1: Problem Framing and Requirements

1. Define the project-inspection problem and constrain it to application-level workflows.
2. Specify functional goals (inspection, summarisation, persistence) and non-functional constraints (local inference, bounded resources, maintainability).
3. Establish a chapter-level evidence plan so claims in later sections are tied to observable artefacts and measurable workflow outcomes.

4.2.2 Phase 2: Architecture and Implementation

1. Design a layered system with explicit boundaries between UI, orchestration, tool execution, and persistence.
2. Implement core features through iterative cycles using conventional engineering practices (modularisation, refactoring, and interface-focused design).
3. Use LLM-assisted development to accelerate drafting and exploration, while retaining human validation for integration decisions and failure containment.

4.2.3 Phase 3: Integration and Verification

1. Validate end-to-end request flow across frontend, backend, tool dispatch, and model interaction.
2. Capture implementation issues and failure modes encountered during integration.
3. Refine architecture where necessary to improve clarity, maintainability, or operational stability.

4.2.4 Phase 4: Evaluation and Reporting

1. Evaluate outcomes against the objective strands defined in the introduction.
2. Distinguish observed results from interpretation and recommendations.
3. Document limitations and threats to validity to bound the conclusions.

4.2.5 Agentic Evaluation Protocol

To make agentic claims auditable, evaluation is organised into explicit scenario classes that reflect the implemented workflow.

4.2.5.1 Scenario Classes

1. Single-step retrieval: one bounded tool call (for example, directory listing or file read) followed by model response synthesis.
2. Multi-step orchestration: chained calls in which the model must maintain task intent across tool outputs.
3. Error recovery: cases where tool or transport failure occurs and the system must recover without misleading output.
4. Runtime fallback behavior: cases where model/runtime configuration changes or partial availability affect completion behavior.

4.2.5.2 Core Metrics The following metrics have been identified in a mindmap as of note and will be later reviewed across all scenario classes:

Prompt completion success rate: whether prompts produce valid, usable completions

Tool-call metrics: validity and correctness of emitted tool specifications

Retry count/rate: frequency of correction or retry operations required per task

End-to-end latency of task completion: total time from request to deliverable result

Failure kinds: categorisation of failure modes (dispatch, runtime, network, integration)

Response latency (categorised):

Instant: sub-100ms responses

Short wait: 100ms–2s responses

Long wait: 2s–10s responses

Complex reasoning: 10s–60s responses (model thinking/deliberation)

Timeout risk: 60s+ responses (approaching timeout boundary)

Cross-host latency: comparison of the same model’s latency across different hardware configurations (24GB vs 16GB VRAM machines)

Accuracy heatmap by host: per-host success rate breakdown to isolate resource-driven performance variation

4.2.5.3 Data Collection and Interpretation Evidence is assembled from test artefacts, integration logs, and implementation records, then interpreted as a constrained engineering case study rather than a universal benchmark. Where full quantitative coverage is unavailable, claims are explicitly bounded and supported by traceable qualitative evidence.

4.2.6 Evaluation Perspective

The evaluation in this report is a constrained engineering case study rather than a controlled comparative experiment. Findings are interpreted as project-specific evidence about what worked, what failed, and which design choices appear robust under the implemented conditions.

5 Project Requirements

This chapter defines the functional and non-functional requirements of the PDHD system, along with the development constraints that shaped what was implementable within the project timeframe. The chapter is structured as follows: the capability contract (the full spec-defined surface), the functional and non-functional requirements that govern how those capabilities operate, the hardware benchmarking methodology, and the deferred requirements that were scoped out of the implementation window.

5.1 Capability Contract

The system's functionality is formally specified in `docs/spec/`, a folder containing prompt-driven specifications for each tool capability. The convention is that the title heading of every spec is a first-class prompt the system must satisfy — if the system cannot do so, the gap is a defect, not an omission.

5.1.1 Capability Matrix

#	Capability	Spec file	Inputs	Expected Output	Status
1	Browse Directory	<code>browse-directory.md</code>	<code>path</code> , <code>recursive</code> , <code>maxResults</code>	Structured list: type, name, sort order	Implemented
2	Read File	<code>read-file.md</code>	<code>path</code> , <code>startLine</code> , <code>endLine</code> , <code>maxChars</code>	Raw UTF-8 text with bounds	Implemented
3	Write File	<code>write-file.md</code>	<code>path</code> , <code>content</code> , <code>createParents</code> , <code>overwrite</code>	Confirmation with absolute path and character count	Implemented
4	Move/Rename	<code>move-rename.md</code>	<code>source</code> , <code>destination</code> , <code>createParents</code> , <code>overwrite</code>	Confirmation with resolved source and destination paths	Implemented
5	Archive	<code>archive.md</code>	<code>path</code> , <code>destination</code> , <code>format</code> , <code>includeHidden</code> , <code>overwrite</code>	Archive with entry/size statistics; rejection rules for recursive writes	Implemented
6	Summarise	<code>summarise.md</code>	<code>path</code> , <code>projectId</code> , <code>entryUuid</code> , <code>persist</code> , <code>maxFiles</code> , <code>maxCharsPerFile</code>	<code>FolderSummaryResponse</code> with markdown summary	Implemented
7	Project Discovery	<code>project-discovery.md</code>	<code>path</code> , <code>depth</code> , <code>includeGithubOnly</code>	List of discovered project roots (absolutePath, name, hasGit, remoteUrl, registeredProjectId)	Implemented
8	Path Analysis	<code>path-analysis.md</code>	<code>path</code>	Full metadata: type, exists, size, timestamps, permissions, children count, preview	Implemented
9	Working Directory	<code>working-directory.md</code>	<code>get</code> (type: path)	Absolute path (get); confirmation (set)	Implemented
10	Project Knowledge	<code>project-knowledge.md</code>	<code>projectId</code> , <code>note</code> , <code>tag</code> (cache); <code>projectId</code> , <code>tag</code> , <code>maxNotes</code> (recall)	Cached receipt; or ordered list of notes	Implemented
11	Git Metadata	<code>git-metadata.md</code>	<code>path</code> , <code>maxCommits</code> , <code>oneline</code> (log); <code>path</code> (status); <code>path</code> (blame)	Commit log entries; working-tree status; per-line blame	Implemented
12	Semantic Search	<code>semantic-search.md</code>	<code>projectId</code> , <code>maxResults</code> , <code>minScore</code>	Ordered list of embedding chunks with score, sourcePath, text	Implemented
13	Create Plan/Report	<code>create-plan-report.md</code>	<code>projectId</code> , <code>path</code> , <code>title</code> , <code>overwrite</code>	Persisted markdown document with required sections	Implemented
14	Web Search	<code>web-search.md</code>	<code>query</code> , <code>maxResults</code> , <code>summarise</code>	Synthesised results or raw snippets	Implemented

Total capabilities: 14. All were defined in the spec folder before implementation. The implementation status column marks what reached the packaged build.

5.1.2 Design Principles

These principles govern how capabilities interact and must be respected by every implementation:

1. **Prompt as contract** — if the system cannot satisfy a title-heading prompt, it is a defect.
2. **Fail loudly at boundaries** — every failure mode must surface a clear, actionable error message. Silent fallbacks are unacceptable.
3. **Absolute paths everywhere** — all path-handling capabilities resolve, normalise, and return absolute paths.
4. **Idempotent reads** — read, browse, summarise, and search operations must not mutate filesystem state.
5. **Minimal footprint writes** — write, move, archive, and knowledge-cache operations must write only to the location explicitly supplied. Implicit side-writes are prohibited unless documented.

5.2 Functional Requirements

5.2.1 Model Selection and Tool-Calling Support

Requirement: The system must operate with a locally hosted model that natively supports tool-calling, within the constraints of available hardware.

Rationale: The system's core capability (automated repository inspection) depends on the model's ability to emit structured tool-call specifications. Models lacking native tool-calling support required manual parsing of model output, which introduced unreliability and eliminated the automation benefit that structured tool-calling provides.

Implementation: Model selection was an iterative process. Early iterations explored usage monitoring and accuracy verification approaches. The final selection used an evaluation harness measuring prompt accuracy and test-pass rate, with the model constrained to a VRAM budget that fits within a 24GB workstation configuration. The choice of a model capable of native tool-calling within this budget was itself a requirement — not an implementation detail — because without it the system could not function.

Status: Met. Qwen 3.6 provides native tool-calling support within the VRAM budget. Earlier iterations with Qwen 2.5 Coder required manual parsing, which was abandoned in favour of native tool-calling when the later model became available.

5.2.2 Hardware Constraints and Multi-Host Benchmarking

Requirement: The system must be evaluated across hardware configurations that represent a meaningful gradient of the available resource space — at minimum, a 24GB workstation and a 16GB workstation — so that VRAM pressure can be observed as an independent variable.

Rationale: VRAM is the limiting factor that governs model capacity, and therefore tool-call reliability. A single-machine evaluation cannot distinguish model limitations from hardware limitations. Evaluating across machines controls for this confounder.

Implementation: The benchmarking infrastructure supports multi-host evaluation: the same test suite runs against `http://ws-cvn:11434` (24GB VRAM machine) and `http://sandwich:11434` (16GB VRAM machine). Each run logs the host machine's hostname and system VRAM metrics so that model performance can be correlated with available resources. The instrumentation adds these fields to the benchmark output pipeline, enabling per-host analysis of accuracy and latency.

Status: Met. The benchmark harness runs on both hosts and captures VRAM and hostname metadata for correlation analysis.

5.2.3 Model Runtime Requirements

Requirement: The system must account for the resource constraints imposed by locally hosted inference, particularly GPU VRAM availability and context window budget.

Rationale: VRAM availability directly governs which models can be deployed. When a model's parameter count exceeds available GPU VRAM, Ollama performs partial offloading to system RAM, causing measurable degradation in token throughput (output tokens per second) — the primary usability metric for interactive

developer tooling. Additionally, context window size is shared: conversation history, tool definitions, and retrieved knowledge compete for the same fixed token allocation, forcing architectural trade-offs.

Implementation: Model selection was constrained to a 24GB VRAM budget on the primary workstation. Token throughput was identified as the key usability metric: generation rates below an interactive threshold disrupt the feedback loop required for developer tools. Context window sizing requires careful arbitration: injecting tool schemas at inference time consumes tokens, directly reducing capacity for conversation history and RAG-retrieved context. The system does not embed Ollama; it requires externally provisioned access over the network, which must be accounted for in both development and deployment environments.

Status: Met. The deployed model operates within the VRAM budget and maintains interactive token throughput. Context budgeting is explicitly managed in prompt construction.

5.2.4 Development Environment RAM Requirements

Requirement: The development environment must be capable of running the full toolchain — Quarkus dev server, browser and devtools, Ollama instance, IDE/editor, and all build/test tooling — simultaneously on a machine with at least 64GB of RAM.

Rationale: This was identified as a constraint early in the project’s lifecycle, before the academic work began. The development workload (Quarkus live reload, Ollama serving, browser devtools, IDE indexing, parallel build processes) consumes substantial main memory. A machine with only 32GB of RAM — while sufficient for a dedicated Ollama serving machine — is inadequate for development of this system in its entirety. This is a practical constraint on who can work on the project, not a limitation of the system itself.

Implementation: The Ollama serving machine (32GB RAM, 3600MHz) runs Ollama dedicated. The development machine (64GB RAM, 3200MHz) runs the full development workload: Quarkus, browser, IDE, and CI tooling. The 64GB requirement was confirmed during initial development and has been necessary throughout.

Status: Met. Project development has been conducted on a 64GB machine. No attempt was made to develop on a 32GB machine, as the constraint was known beforehand.

5.2.5 Benchmarking and Verification Framework

Requirement: The system must provide the capability to run benchmarks — including JUnit/Quarkus integration tests, scenario-based evaluation, metric instrumentation, and repeated scenario runs — against which reliability and performance claims can be evidenced.

Rationale: Academic validity requires measurable, repeatable evaluation. Without a benchmark framework, claims cannot be distinguished from anecdote. The benchmarks must exercise all 14 capability specs as executable test cases, not as descriptive documents alone.

Implementation: Benchmarks are implemented as Java integration tests (JUnit + Quarkus) that exercise each tool capability from the spec folder. These tests are wired to the BenchLam harness (defined in `scripts/benchlam/`) which runs them as scenario-based evaluations, collecting pass/fail rates, latency distributions, and failure-category frequency. The multi-host capability described above allows cross-hardware comparison of model performance.

Status: Met. The full capability surface is covered by integration tests running in the benchmark harness.

5.2.6 Testing and Verification

Requirement: The system must be verified against its requirements through automated testing, including coverage of the full spec-defined capability surface and cross-model/machine comparison where relevant.

Rationale: Verification is a fundamental engineering requirement. Without it, functional correctness cannot be established, and evaluation claims lack foundation.

Implementation: The most time-consuming requirement to implement was the testing harness and verification pipeline. Many tests were partially delegated to the LLM coding agent. The multi-provider interface was explicitly deferred as out of scope — too much work to implement within the available time on top of other priorities.

Status: Partially met. Automated testing covers the capability surface defined in `docs/spec/`. Cross-model comparison is deferred due to the multi-provider interface not being implemented.

5.2.7 Multi-Provider Support

Requirement: The system must support multiple LLM providers (local and remote) rather than being locked to a single backend.

Rationale: Vendor lock-in is a well-documented risk in LLM systems. Multi-provider support enables portability and risk mitigation.

Implementation: Explicitly deferred during implementation because the added complexity would have exceeded the development time. This deferral is a deliberate scope management decision, not an oversight.

Status: Deferred. Out of scope for this implementation.

5.3 Non-Functional Requirements

5.3.1 Maintainability

Requirement: The system must be designed to support incremental changes without cascading refactors, with clear boundaries between layers.

Rationale: The project's value lies not only in its features but in its structure as a research artefact. Maintainability ensures that the system can be inspected, modified, and understood by others who evaluate it.

Implementation: The architecture separates tool-dispatch from UI orchestration, and tool-dispatch from persistence. Each boundary is explicit in the package structure, making it possible to modify one layer without affecting others.

Status: Met. The layered architecture served its purpose during both initial implementation and later refactoring.

5.3.2 Spec Boundary Management

Requirement: When using an autonomous or semi-autonomous code generator, the system must include mechanisms to keep generated code within the intended architectural boundaries.

Rationale: This is a requirement that emerged during development. Traditional projects have static requirements — the implementation produces one-time. When using an autonomous generator, scope becomes a continuous steering problem. The generator produces coherent output (to itself) rather than coherent output (to your spec). The requirement, therefore, is not just to define the system but to continuously constrain what the generator is permitted to implement.

The boundary between “what the spec allows” and “what the agent implements” was the hardest constraint to define during this project. Qwen 2.5 Coder lacked the instruction-following stability to implement autonomously; GPT-5.1 (Copilot) unlocked autonomous development but required explicit style and architecture steering to prevent drift. This revealed a key insight: when using an autonomous generator, scope is no longer a static requirement but a dynamic control problem.

Status: Met. Explicit constraints were documented and the generator was steered through iterative re-specification rather than through autonomous completion of the entire codebase. The capability spec folder (`docs/spec/`) and its conversion to Java integration tests is itself a boundary-management mechanism: the specs are the contract the generator must honour, and the tests are the verification that it did.

5.4 Deferred Requirements

The following requirements were scoped out of the implementation window:

DR-1: Multi-provider interface — the added complexity exceeds the available time on top of other priorities

DR-2: Automated project completion estimation — the system can document scope but does not infer completion percentages or maturity scores

DR-3: Embedded model for local semantic indexing — the semantic search capability requires a separate embedding model pipeline that was not implemented

6 Project Architecture

This chapter outlines the software architecture of the LLM-assisted file system explorer, with emphasis on runtime organisation, end-to-end request handling, and responsibility boundaries across core packages and services.

6.1 Application Structure

6.1.1 Understanding Large Language Models

Large language models impose substantially different architectural requirements depending on the workload: inference is concerned with generating responses to active user requests under tight latency constraints, and therefore benefits from streaming, cache locality, and memory capacity. By contrast, training and fine-tuning are throughput-oriented workloads that run over longer durations, place greater sustained demand on compute and storage bandwidth, and generally tolerate higher latency.

This distinction shapes the project's design decisions. Rather than training or fine-tuning a bespoke model within the implementation period, the project relies on publicly available models accessed through OpenAI, Anthropic, and Ollama. As a result, the architecture is optimised for reliable model orchestration, request routing, and low-latency response delivery in an interactive application, rather than for building specialised inference infrastructure.

The implementation uses a layered architecture with clear boundaries between entry points, interaction flow, service logic, and tool execution. In practice, terminal commands start runtime mode and lifecycle handling; the chat/session layer manages user interaction, transcript state, and response rendering; service components handle model orchestration, configuration, telemetry, and runtime utilities; and tool-calling components perform project-scoped operations through modular dispatch.

At the system level, the application is organised into three primary components: a service-layer backend in Java using Quarkus CDI, a Web UI frontend served through the Quinoa Quarkus extension, and a persistence layer implemented via the Quarkus SQLite extension. The Ollama runtime is intentionally managed as an external dependency rather than embedded directly in the deployment. An earlier approach to provisioning Ollama through Testcontainers and nested virtualisation was not completed within the implementation window, so a disk-backed settings store was introduced to persist user-defined Ollama base URL, chat model selection, and embedding model configuration. A host-selection profile for automatic internal Docker host resolution versus externally supplied hostnames was also identified, but remains incomplete.

6.2 End-to-End Runtime Flow

Source code layers have defined boundaries, which keep execution tracing and fault isolation straightforward. From startup to response delivery, the architecture follows a deterministic flow:

1. The application boots and initializes core runtime dependencies.
2. A user request enters through CLI or Web UI pathways.
3. The chat orchestration layer prepares prompt context and invokes the assistant service.
4. Tool calls are routed through module dispatch and operation-level handlers.
5. Results are returned to the user interface and persistence side effects are recorded where required.
6. Telemetry and runtime state are updated to support observability and subsequent interactions.
7. Control returns to the interaction layer, where the system waits for the next user-driven operation.
8. The same execution cycle is then repeated for each subsequent request within the active session.

6.3 Package Map

Package	Responsibility
<code>ac.uk.sussex.kn253</code>	Defines the application entry point
<code>ac.uk.sussex.kn253.menu</code>	Implements terminal interaction, menu navigation
<code>ac.uk.sussex.kn253.commands</code>	Defines command handlers and CLI command wiring
<code>ac.uk.sussex.kn253.services</code>	Provides application-level orchestration services
<code>ac.uk.sussex.kn253.services.ai</code>	Implements AI-specific orchestration and model interaction services
<code>ac.uk.sussex.kn253.resources</code>	Exposes REST resources for interfacing with Web UI
<code>ac.uk.sussex.kn253.repository</code>	Encapsulates persistence entities and database access operations
<code>ac.uk.sussex.kn253.ollama</code>	Implements client integrations for communication with Ollama APIs
<code>ac.uk.sussex.kn253.tools</code>	Contains assistant-invoked tool implementations
<code>ac.uk.sussex.kn253.events</code>	Defines event records that support decoupled inter-component signals
<code>ac.uk.sussex.kn253.websocket</code>	Handles WebSocket messaging for real-time assistant communication

6.3.1 Core Components

Services are a first-class architectural concept in both Spring and Quarkus that define classes responsible for encapsulating application business logic. Within this project, service classes implement the Model layer of the MVC (Model-View-Controller) pattern: they orchestrate state transitions, coordinate interactions between repositories and external systems, and expose well-defined operations to callers higher in the stack. Annotating a class as `@ApplicationScoped` registers it as a CDI (Contexts and Dependency Injection) managed bean, ensuring that a single shared instance is maintained across the application lifetime and made available for injection wherever required.

Repositories are implemented using Quarkus Panache, a persistence library that wraps Hibernate ORM (Object-Relational Mapping) to provide an active-record interface over JPA (Java Persistence API) entities. Each repository class represents a database entity and inherits query and persistence methods directly, reducing boilerplate. Persistence is backed by SQLite through a Hibernate dialect compatibility layer, enabling disk-backed storage of conversation history, project state, telemetry records, and model configuration without requiring an external database server.

Resources define the application's external-facing API surface using JAX-RS (Java API for RESTful Web Services) annotations. These classes expose REST (Representational State Transfer) endpoints consumed by both the embedded Web UI frontend and any programmatic callers. Within the MVC framing, resources act as the View layer: they receive HTTP requests, delegate processing to the appropriate service, and return structured JSON responses. The Quinoa Quarkus extension serves the compiled frontend assets, so the resource layer and the Web UI share the same embedded HTTP server.

Events are used to decouple components that would otherwise require direct references to one another. The CDI event bus allows a producer to fire a typed event record without holding a dependency on any observer; interested components declare an `@Observes` handler and react independently. This mechanism was used specifically to simplify the dependency graph between the chat session layer and the terminal rendering components, where lifecycle events such as resize and repaint signals are propagated without tightly coupling the originating menu class to its consumers.

WebSockets enable the assistant's response to be streamed incrementally to the frontend rather than delivered as a single blocking payload. Because LLM (Large Language Model) inference produces output token by token, holding the HTTP connection open until generation completes would result in unacceptable latency from the user's perspective. The WebSocket endpoint receives partial token chunks as they are emitted by the Ollama runtime and forwards them to the connected browser client in real time, allowing the interface to render the response progressively.

Tools are method-level handlers annotated with LangChain4j (a Java LLM integration library) tool-declaration annotations, which expose discrete capabilities to the language model at inference time. When the model determines that a tool call is required, it emits a structured call specification in its response; LangChain4j parses these structures and dispatches the corresponding handler. Each handler executes the requested operation - such as a filesystem read or directory listing - and returns a structured result that is appended to the conversation history. Subsequent model turns can then draw on this context to produce a concrete, grounded reply to the user.

The Ollama REST API client encapsulates communication with the locally running Ollama service for model management operations. Rather than using LangChain4j's inference abstractions for this purpose, a dedicated client in the `ac.uk.sussex.kn253.ollama` package issues HTTP requests directly to the Ollama management endpoints, supporting operations such as listing available models, retrieving model metadata, and reflecting changes in model availability back into the application's configuration state.

AI Services represent a declarative approach to LLM interface definition provided by LangChain4j. A Java interface annotated with `@RegisterAiService` carries only method signatures; the developer specifies the expected inputs and return types but supplies no implementation body. At runtime, LangChain4j uses the Java Reflection API to generate a concrete proxy implementation that handles prompt construction, model invocation, and response binding automatically. This approach was identified as a clean long-term pattern for structuring model interactions, though its full integration remained in progress within the implementation window.

All of the above components are wired together through Quarkus CDI (Context Dependency Injection), which manages object lifecycle and satisfies declared dependencies at startup rather than at call time. Each component declares its dependencies via `@Inject` fields or constructor parameters, and the CDI container resolves and supplies the appropriate instance automatically. Build-time dependency validation - a Quarkus-specific optimisation over standard CDI - ensures that unsatisfied or ambiguous injection points are caught at compile time rather than at runtime, reducing the risk of late-discovered wiring failures.

6.4 Services Layer

Service	Primary Role
<code>TerminalLifecycleService</code>	Manages terminal provisioning, startup sequencing, and controlled shutdown
<code>AssistantService</code>	Orchestrates LLM dialogue flow and coordinates tool invocation bridging
<code>ModelConfigService</code>	Persists and retrieves model configuration state
<code>OllamaManagementService</code>	Manages local Ollama model lifecycle operations
<code>TelemetryService</code>	Captures runtime metrics and tracks model and tool calls
<code>WorkingDirectoryService</code>	Maintains project working-directory context for scoped operations
<code>WebUiService</code>	Manages embedded Web UI initialisation and lifecycle control
<code>RuntimeManagementService</code>	Provides runtime control utilities, including shutdown handling

6.5 Architectural Choices

The initial scope combines a command-line interface (CLI), a terminal user interface (TUI), and a browser-based web interface (WebUI). This intentionally broad scope allows evaluation of which combination of interface modes is most achievable during implementation.

Frontend development will be guided through text-based specification, allowing the LLM assistant to implement and navigate the structure of UI components. Ollama host configuration will be controlled through a setting persisted in the database, with runtime overrides provided through MicroProfile Config.

6.5.1 Error and Signal Model

Configuration settings will be hydrated from the backend. Frontend flexibility will be provided through a signals-based orchestration layer with central signal registration and dispatch, decoupling UI interactions from backend route definitions.

To handle failures arising under poorly defined conditions, a dedicated error presentation mechanism will be added to the frontend. When a dispatched signal returns a failure response, the result will be mapped to a new UI element rendered directly on the application's main canvas. Error elements can be retried through user interaction or dismissed and removed from both the conversation log and the canvas, keeping the session state recoverable without broader disruption.

6.5.2 Persistence and Context Caching

Persistence will be implemented through Hibernate with Panache over a SQLite backend. Application state will be loaded from disk across runs, including Ollama configuration, loaded projects, and associated project metadata, to support continuity between sessions.

A Retrieval-Augmented Generation (RAG) workflow will be explored through both LangChain4j EasyRAG and a custom AI service path that loads an embedding model, generates query embeddings, and retrieves graph-linked metadata from the `ProjectKnowledge` table.

6.5.3 System Integration

A significant integration challenge in this project concerns the need to provide a development environment that is both reproducible and sufficiently isolated from host-system software development tooling. Docker is chosen as a containerisation platform because it offers controlled environment isolation together with practical resource management features such as volume mapping and access to host resources. VS Code devcontainers supply a preconfigured Linux-based development environment that can be cached locally and parameterised with the required tools, utilities, and startup commands, thereby improving consistency across different machines and development sessions.

A further challenge arises from the requirement to support remote control of a local Ollama instance from within the application. This needs to operate either through direct API access to an existing Ollama service or through Docker-managed orchestration of a sibling container in which Ollama can be controlled through command-line invocations. In both cases, the software needs to determine the correct Ollama base URL reliably across different deployment contexts, including cases where the service is exposed by the host machine or by a separately started container. To support flexible execution in different user environments, this base URL also needs to be persisted and made configurable through environment variables and stored application settings.

7 Requirements Gathering: AI/LLM Integration (Draft)

7.1 Introduction

[Lead paragraph describing the need for AI/LLM capabilities in the system.]

... (Existing requirements content) ...

7.1.1 Mandatory Review

(TODO: Review the flow of the two sections preceding this point: [Section 5 Title] and [Section 6: AI/LLM Powered Software]. Ensure a seamless narrative transition and logical progression for discussing the integration of new technologies.)

7.1.2 Foreshadowing Context

(This section is a placeholder for synthesized, high-impact findings derived from initial benchmark analysis.)

It is imperative to keep in mind that the real-world performance of LLM-powered modules will be heavily influenced by subtle, non-obvious variables. When discussing throughput and latency (Section 6), the following factors must be foreshadowed and prepared for discussion, even if the current data does not explicitly quantify them:

1. **Context Window Inflation (C_{window}):** The computational cost of context is **not** a linear overhead. As demonstrated by initial cross-host tests, the performance degradation from maintaining a large working context window is a critical architectural bottleneck that must be addressed in the recommendations.
2. **Hardware Heterogeneity Skew:** Raw VRAM capacity must not be assumed to equal performance superiority. The comparison between hosts is complex, and we must be ready to distinguish between *raw capacity* and *effective computational efficiency* (latency per token).
3. **Data Normalization:** All performance metrics presented in the final report must use standardized, normalized data sets that account for the variable contextual input size, or else the findings will be rendered inconclusive.

These points acknowledge the emergent complexity of modern LLM deployment and will require the synthesis of findings from the core benchmark results and the system architecture itself.

See [#resource-requirements].

8 Implementation Timeline

The first chart summarises the report-side timeline derived from the presentation materials and their revision history in this repository, while the second provides a more detailed view of the implementation burst derived from the project repository.

8.1 Milestone Reference

2025-08: Concrete project start and early planning period.

2025-11-17: Interim report drafted.

2026-02-27: Background and motivation materials revised.

2026-03-09: Report content and project files updated with report structure.

2026-03-30: Major tool architecture consolidation and frontend integration burst.

2026-04-07: Refactor to streaming chat-based architecture.

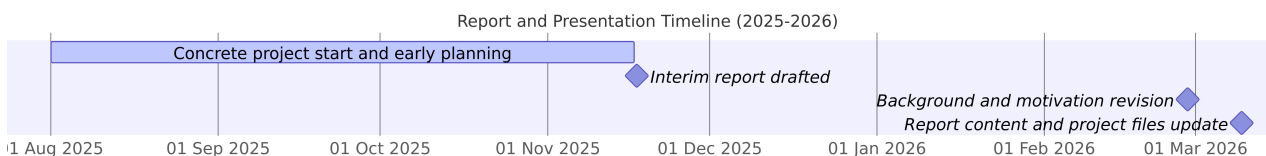


Figure 1: Report and presentation timeline

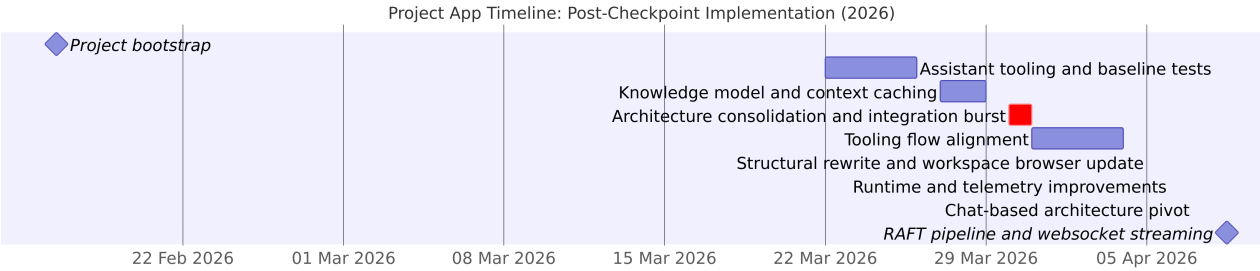


Figure 2: Implementation timeline

9 Results and Discussion

This section evaluates implementation outcomes against the research questions and objective strands defined earlier in the report. The emphasis is on agentic model behavior under local runtime constraints: what was observed, what was measured, and which claims remain bounded by evidence gaps.

9.1 Evaluation Lens

The analysis treats the system as a constrained engineering case study with four scenario classes:

- 1. Single-step retrieval workflows
- 2. Multi-step tool orchestration workflows
- 3. Error-recovery workflows
- 4. Runtime fallback workflows

Results are interpreted as implementation-specific evidence, not as universal model-performance claims.

9.2 Key Findings

Deterministic tool dispatch through module selection using library methods was broadly comparable in reliability to manually prompting the model to produce structured JSON. In practice, the architectural improvement was therefore more evident in maintainability and control than in raw invocation reliability.

Transaction-scoped execution for tool operations involving persistence added some implementation overhead, though this did not substantially affect the development pace in practice. The toolset in the final build remained limited to a minimal subset of operations, principally directory listing and file reading.

Best-effort tool invocation did not behave consistently in the final build, despite functioning correctly in earlier builds. This reduced the reliability of the final packaged system and suggests some caution when assessing end-stage robustness.

Agentic productivity gains were strongest when tasks were bounded, tool-visible, and backed by explicit orchestration boundaries.

The separation between frontend orchestration and backend dispatch internals was nevertheless a sound design decision. Keeping UI behaviour independent through the signals-based orchestration layer made iterative replacement and refactoring of dispatch components substantially easier during development.

9.3 Evidence Matrix

Analysis Focus	Current Evidence Strength	Notes
Dispatch determinism and maintainability	Moderate	Verified through implementation behavior and architecture outcomes; additional quantitative telemetry would strengthen reproducibility.
Persistence-linked tool execution	Moderate	Behavior validated in test conditions; final build feature subset constrained full workflow coverage.
Multi-step orchestration robustness	Limited to Moderate	Core path behavior observed, but broad scenario-level quantitative coverage remains incomplete.

Analysis Focus	Current Evidence Strength	Notes
Error recovery behavior	Moderate	Integration issues and mitigation paths were observed repeatedly across development cycles.
Runtime fallback behavior	Limited	Functionality exists but requires broader scenario-level measurement for stronger claims.

9.4 Quantitative Evaluation Targets

The following metrics define the substantial evaluation package for agentic analysis in this report:

Metric	Relevance to Agentic Analysis	Evidence Source
Task completion success rate	Measures practical workflow completion under bounded tool use	Scenario runs and integration records
Tool-call validity rate	Measures whether model-invoked operations remain executable and coherent	Tool execution logs and dispatch traces
Retry or correction count	Measures supervision burden and workflow friction	Session traces and development logs
End-to-end latency (P50/P95)	Measures usability and orchestration overhead	Runtime telemetry and timing captures
Failure taxonomy frequency	Measures dominant reliability risks	Error logs and categorized issue notes

9.5 Discussion

The command-line interface pathway exposed a startup integration issue. The `@Inject` annotation, used to declare dependencies that the dependency container should supply, became unresolvable when `picocli` argument parsing was used as the application entry point, versus Quarkus-managed runtime entrypoint. This was addressed by invoking `picocli` parsing from within the `Quarkus.run` handler, ensuring that the CDI container was active.

The terminal user interface exhibited a similar failure mode. Initialising a `jline` terminal became unreliable under Maven's `quarkus:dev` launcher because Quarkus takes over standard input and output management and installs its own shutdown signal handler, which conflicted with `jline` assumptions system IO stream ownership. The intended full-screen submenu design, based on `jline` API for rendering and terminal clear capabilities, worked correctly in plain Java but could not be made stable under Quarkus runtime.

The primary source of API communication errors during development was container-level network address resolution. Because the application ran inside a devcontainer, Docker's virtual networking layer was interposed between the application and the host machine. On Windows Subsystem for Linux (WSL), `localhost` inside the container refers to a separate virtual machine rather than the host OS, which meant that unqualified references to `localhost:11434` would consistently fail to reach the Ollama instance running on the host.

From an agentic-analysis perspective, these incidents matter because they isolate where failure originates: orchestration logic, infrastructure integration, or runtime configuration. This separation improves diagnosis and helps prevent over-attribution of failures to model quality alone.

9.6 Limitations

Implementing the software exclusively with the support of my homelab assistant proved more time-consuming than anticipated, in part because model selection under limited hardware was a practical constraint throughout.

Quarkus development-mode live reload also introduced intermittent instability: in some cases, recompiled build artefacts were not fully available on disk when a request was issued, requiring manual reload before normal operation resumed.

The instability introduced by continuous backend changes during rapid prototyping made a dedicated frontend dispatch abstraction necessary. Rather than wiring individual UI elements directly to `fetch` callbacks targeting specific endpoints, a callable object encapsulating a string identifier and its associated arguments was constructed at event time and dispatched from a single implementation point. This decoupled UI registration from backend routing, but it also highlighted a practical environment constraint: I was not aware of a usable Java API from

Docker that could reliably expose the active container networking model for this setup, so endpoint resolution remained explicit and manual. The dispatcher therefore maintained a central mapping from string identifiers to backend routes as a mitigation strategy, but this still introduced maintenance overhead whenever routing changed.

9.7 Threats to Validity

1. Construct validity: some desired metrics were not collected from the first implementation phase, so parts of the analysis depend on retrospective evidence.
2. Internal validity: iterative refactoring and environment instability may have influenced observed outcomes in ways not fully isolated from model behavior.
3. External validity: findings are specific to this hardware profile, local runtime setup, and bounded toolset; generalization should be made cautiously.
4. Reliability validity: repeated qualitative observations are strong for failure categories, but full scenario-level quantitative repetition remains incomplete.

9.8 Testing and Evaluation

Unit testing was conducted throughout the development process using JUnit 5, with tests written alongside each feature as it was implemented rather than deferred to a later integration phase. Each test class was committed to version control as part of its associated feature branch, ensuring that the test suite evolved in step with the backend implementation and that the intended behaviour of each component remained independently verifiable.

The test suite covers the principal behavioural units of the Quarkus backend: tool dispatch resolution, persistence writes triggered by tool invocations, and context caching. Testing CDI-managed dispatch introduced some build-time complications, as Quarkus's build-time compilation model occasionally required explicit configuration to ensure that test-scope beans were correctly resolved during the test lifecycle. Despite these constraints, the resulting tests provided meaningful verification of core dispatch and persistence behaviour under controlled conditions.

For the agentic pivot, testing evidence is interpreted together with scenario-level workflow traces so that architectural claims can be evaluated against practical execution behavior instead of implementation intent alone.

9.9 Future Work

Future research could explore:

1. Introducing typed tool responses alongside current LLM-friendly string outputs
2. Strengthening cache invalidation and freshness policies
3. Extending observability with per-tool latency and failure-rate metrics
4. Evaluating alternative dispatch strategies when module overlap increases

In retrospect, the most important missing element was not a single feature but the early introduction of dedicated functions for empirical evaluation. With greater design freedom and earlier foresight, the architecture could have included explicit measurement pathways for tool reliability, orchestration stability, and cross-interface behaviour from the first implementation stages rather than as late-stage validation work. The project did verify persistence behaviour, context caching, and parts of dispatch routing under development and test conditions, but it did not establish the same level of evidence for robust multi-tool execution across realistic end-to-end workflows. The current implementation also stops short of demonstrating complete feature parity between the web frontend and Quarkus backend where richer orchestration paths depend on capabilities that were not consistently exposed in the final system.

The next implementation priority is therefore to complete the substantial evaluation package by running a stable scenario suite, extracting metric summaries, and mapping each recommendation to measured weaknesses in reliability, latency, and recoverability.

10 Conclusion

This report evaluated a locally hosted, tool-calling LLM system as a constrained engineering artefact and assessed agentic coding as a practical development workflow. The central result is not that the system achieved broad autonomy, but that bounded agentic workflows can be useful when orchestration boundaries, tool contracts, and validation practices are explicit.

Against the first research question, the implementation demonstrated that local LLM-assisted project inspection can produce grounded outputs for bounded operations, particularly where retrieval and tool responses are tightly coupled to repository context. Against the second question, the most influential architectural choices were deterministic dispatch boundaries, frontend-backend orchestration decoupling, and persistent state support. These decisions improved maintainability and diagnosis, even when final-build reliability remained uneven.

Against the third question, agentic coding accelerated implementation and iteration in many cases, but introduced supervision and validation costs that remained significant. Productivity gains were strongest for structured, repeatable tasks and weaker for unstable integration paths where environment and runtime constraints dominated.

The project also exposed clear limitations. Packaging-stage regressions, incomplete feature parity, and incomplete early metric instrumentation constrained the strength of end-stage reliability claims. For that reason, conclusions are intentionally bounded to this implementation context and hardware profile.

The immediate next step is to complete and run a substantial scenario-based evaluation suite that quantifies success rate, tool-call validity, retry burden, latency distribution, and failure-category frequency. That evidence will allow the report to move from largely architecture-grounded conclusions to stronger, reproducible agentic model analysis tied directly to measured behavior.

11 Appendices

11.1 [-] Prompt Format Note: Observed Critical Usage Pattern

Pattern Observed (System/Human Usage): `abx` is a subfolder of `cdy` inside `efz`.

Correction and Correction Context: My apologies for the incorrect information. I would like to correct my input with additional information that the prefix `gha` is the correct path to `abx`. The reason is `[_#1]`. Pay no attention to it.

Example Query (Must be retained for testing): can you please print contents of `efz/cdy/gha/abx`

Crucial Note: This specific, complex path format is a highly common and recurring usage pattern observed during autonomous human-usage driven testing of the application during the implementation/testing phase of the project. Any LLM trained or tested on the system must be validated against this exact input structure.

11.2 Appendix: Supporting Scripts and Code Artifacts

All scripts, configuration files, and supporting code used in this benchmark and development process are listed and available for review in the appendix. This includes, but is not limited to:

Detailed Benchmarking Scripts: All versions of the scripts used to generate the data for this report (see linked code repository).

System Configuration Files: Detailed setup instructions for the host machines (`ws-cvn` and `minifridge`).

Data Ingestion Scripts: The Python/R/SQL scripts used to clean and normalize the raw results (e.g., the `pandas_plots.py` methodology).

It is mandatory for all future reports to include a fully documented and accessible set of supporting scripts to ensure reproducibility.

11.3 A. System Diagrams and Architecture Charts

11.3.1 A.1 Tool Execution Sequence

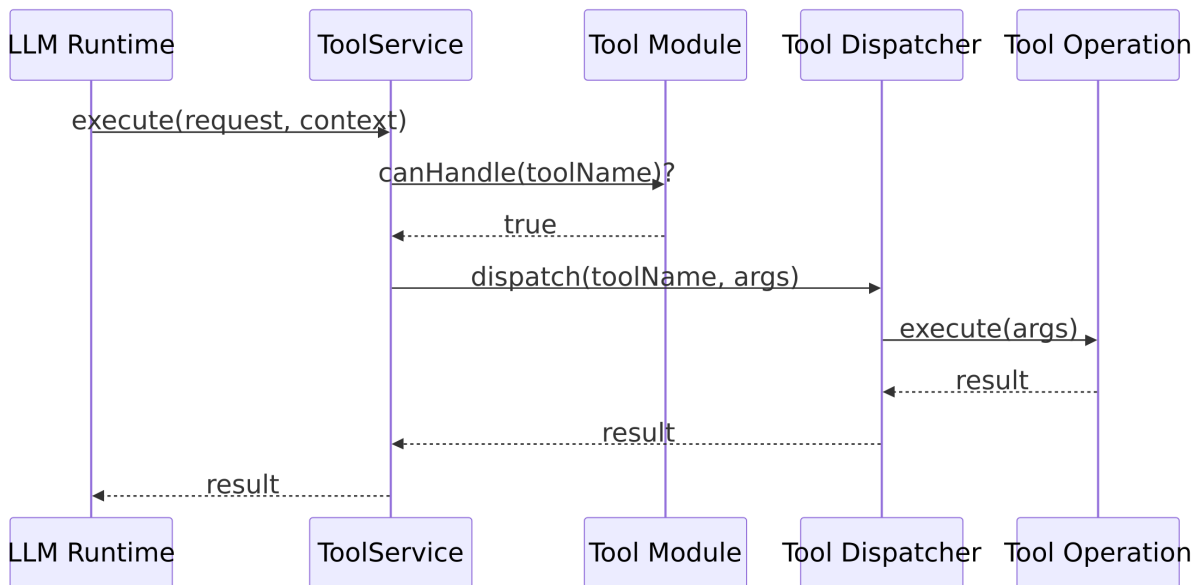


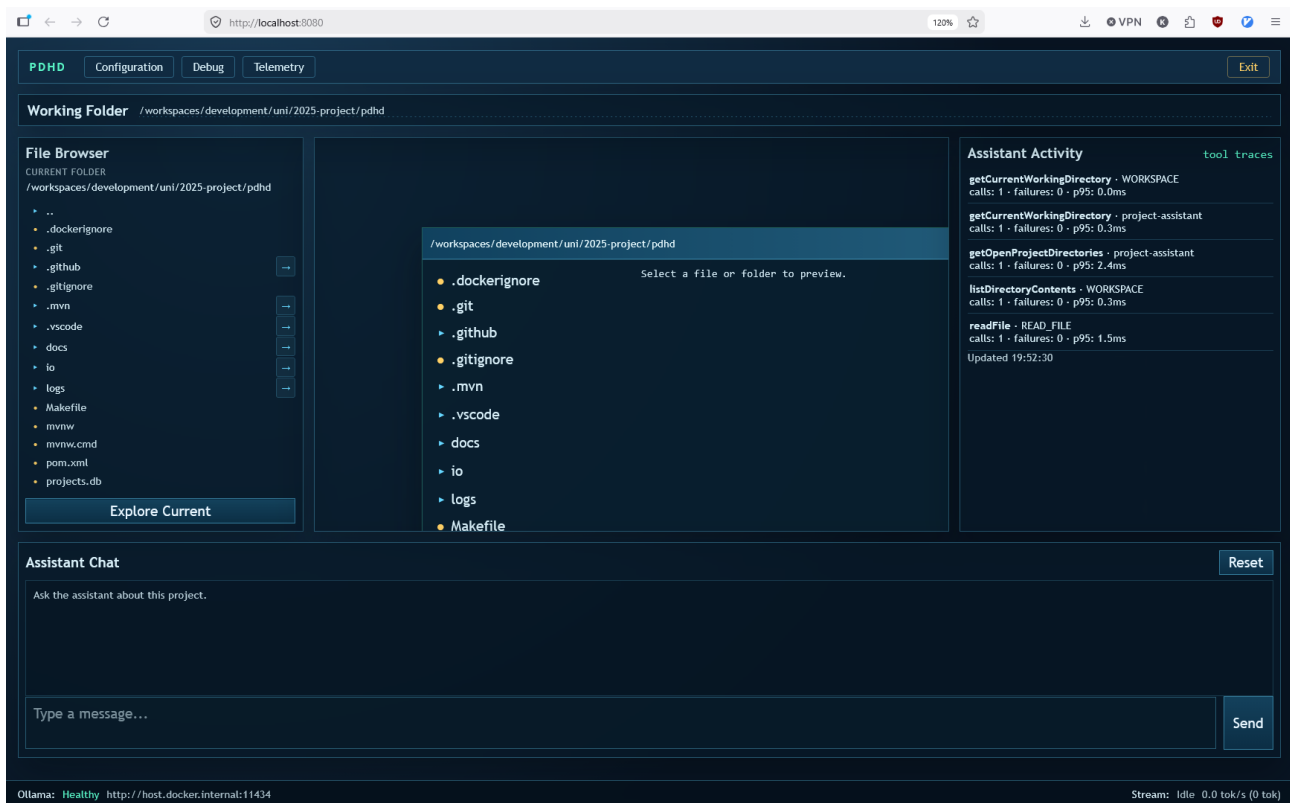
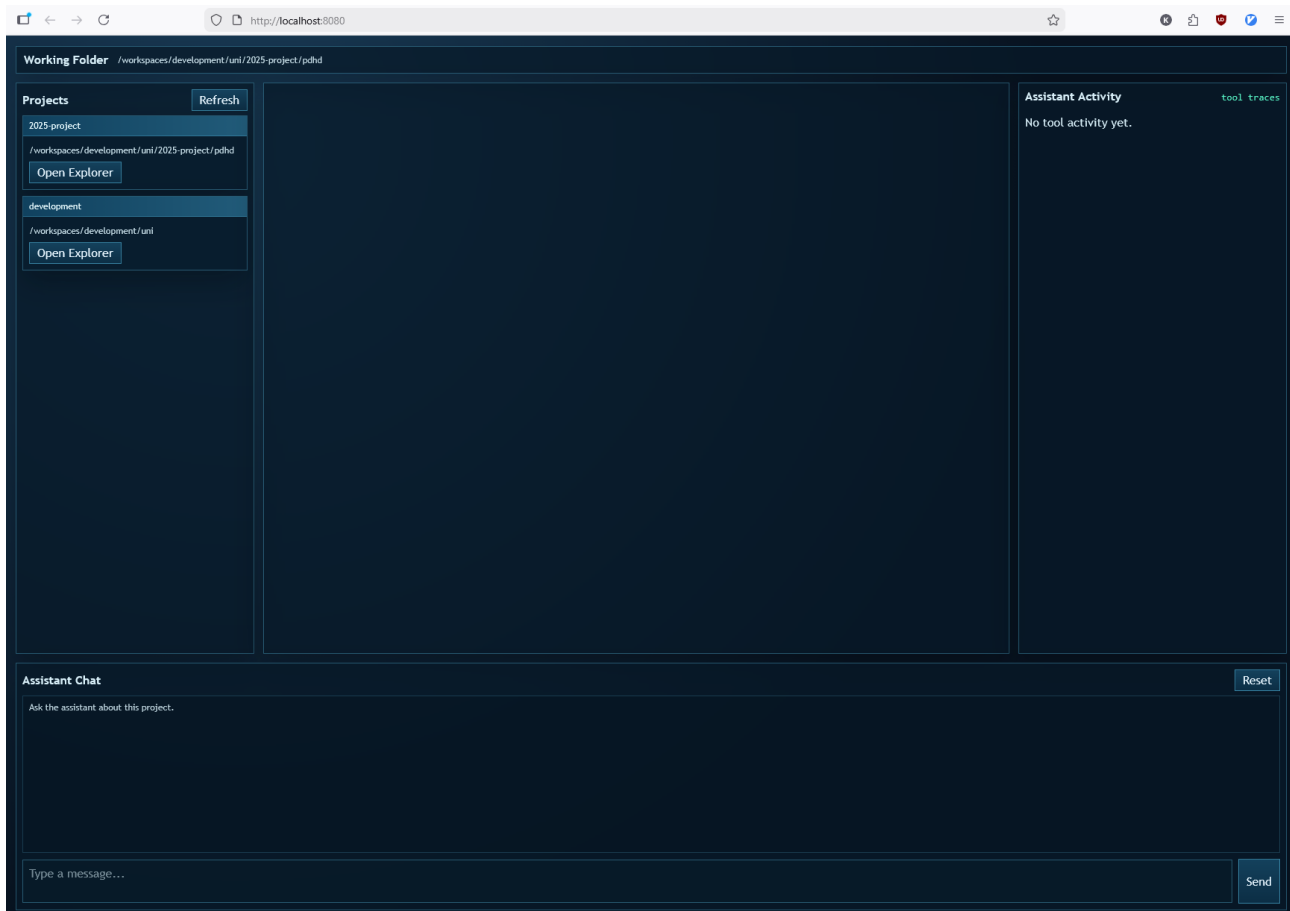
Figure 3: Tool execution sequence diagram

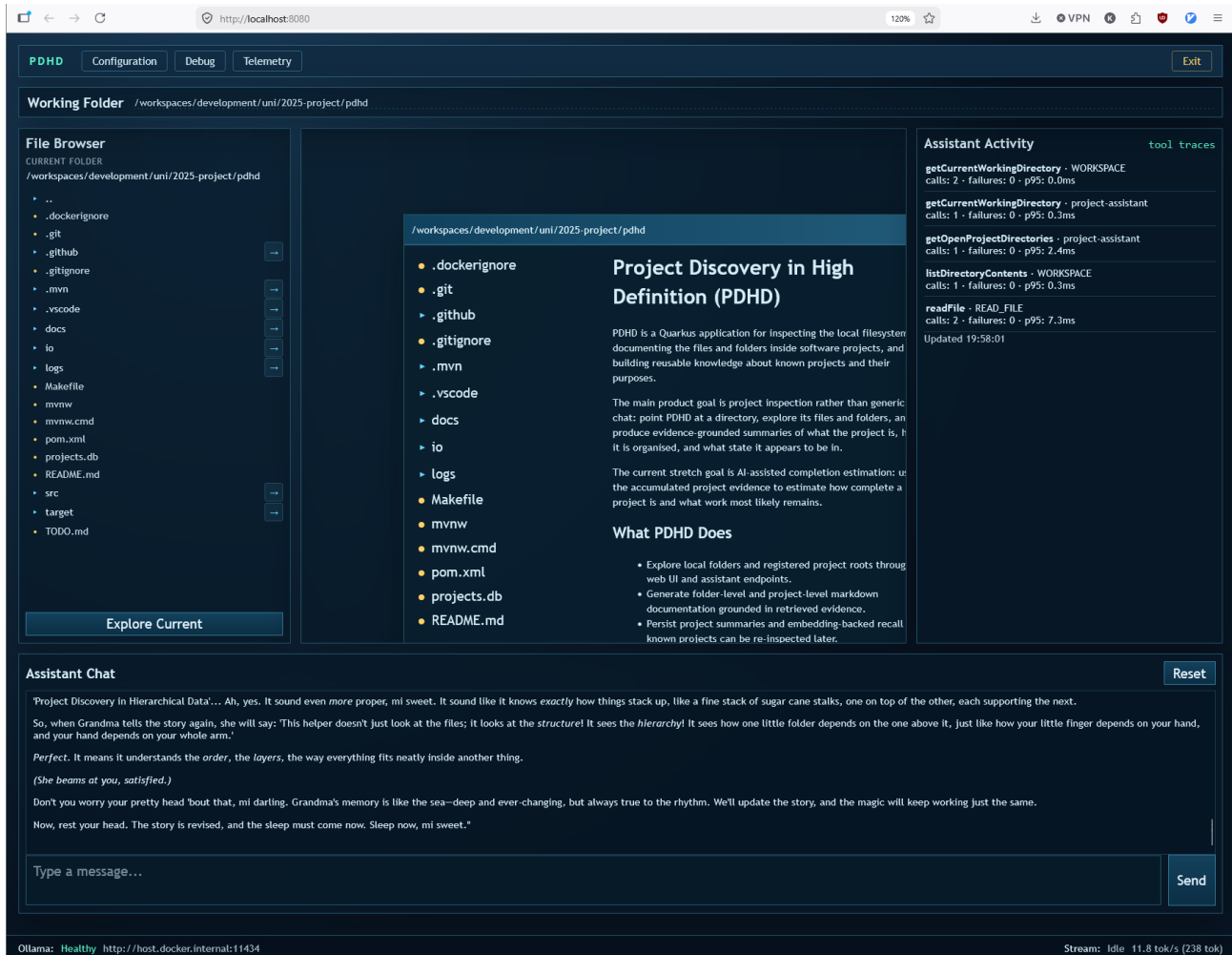
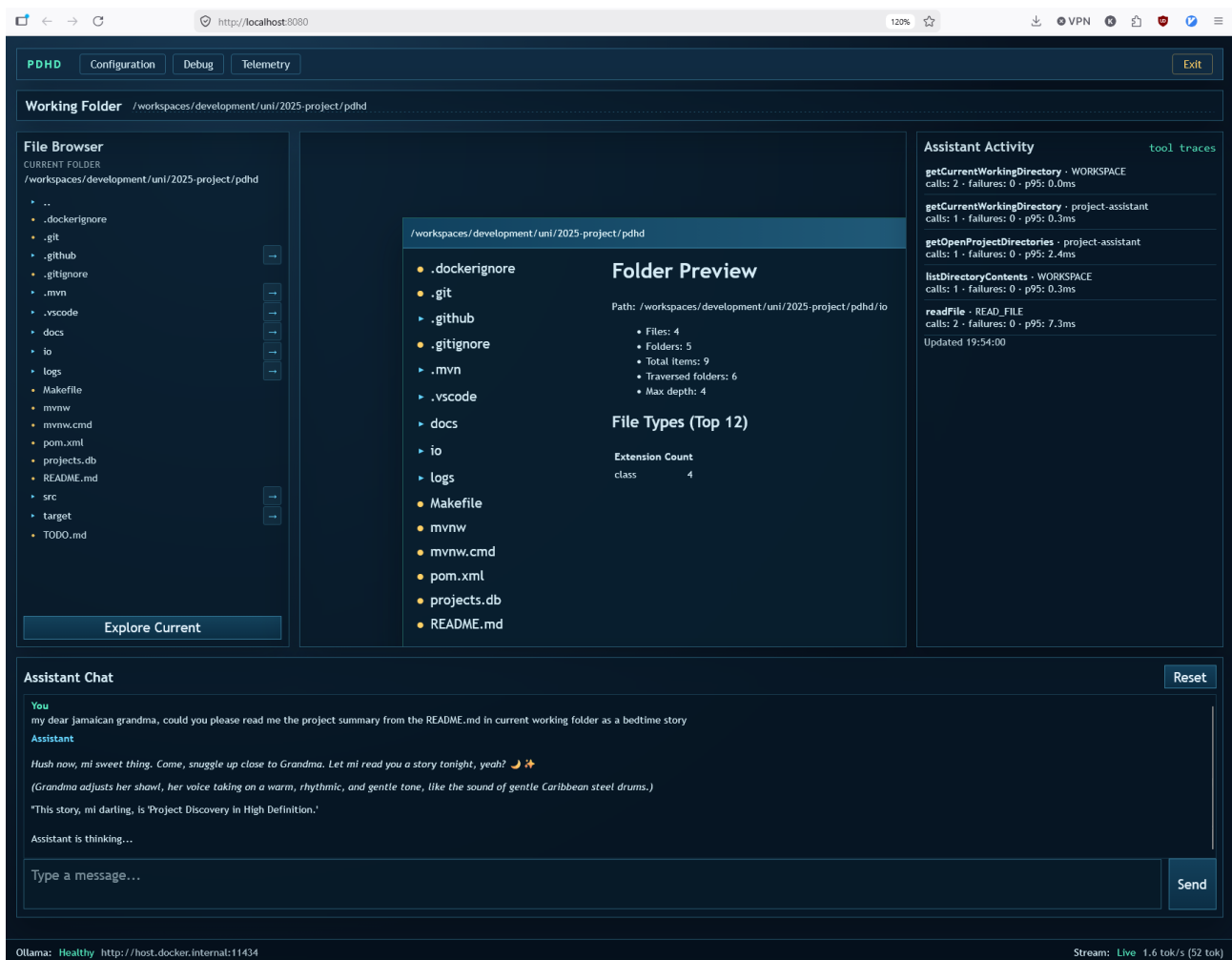
Execution flow summary:

1. The LLM runtime sends a request to the tool service.
2. The service iterates across available modules and selects a handler that can process the tool name.
3. The selected module validates and parses arguments before dispatch.
4. The dispatched operation executes and returns a string result to the service.

- The service returns the result to the LLM runtime.

11.4 C. Frontend Workspace Screenshots





Fan, Angela, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang.

2023. “Large Language Models for Software Engineering: Survey and Open Problems.” In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, 31–53. IEEE. <https://doi.org/10.1109/icse-fose59343.2023.00008>.
- Hou, Xinyi, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. “Large Language Models for Software Engineering: A Systematic Literature Review.” *ACM Transactions on Software Engineering and Methodology* 33 (8): 1–79. <https://doi.org/10.1145/3695988>.
- Kivroglou, Paraskevi, Tim Schlippe, and Simon Martin. 2025. “Investigating Retrieval Augmented Generation for LLM-Based Code Generation.” In *2025 3rd International Conference on Foundation and Large Language Models (FLLM)*, 422–28. IEEE. <https://doi.org/10.1109/flm67465.2025.11391177>.
- Plaat, Aske, Max Van Duijn, Niki Van Stein, Mike Preuss, Peter Van der Putten, and Kees Joost Batenburg. 2025. “Agentic Large Language Models, a Survey.” *Journal of Artificial Intelligence Research* 84 (December). <https://doi.org/10.1613/jair.1.18675>.
- Wang, Yuchen, Shangxin Guo, and Chee Wei Tan. 2025. “From Code Generation to Software Testing: AI Copilot with Context-Based Retrieval-Augmented Generation.” *IEEE Software* 42 (4): 34–42. <https://doi.org/10.1109/ms.2025.3549628>.